МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"

ЛЕКЦІЯ 8 (частина 2). КОМП'ЮТЕРНИЙ ЗІР У РОЗРОБЦІ ІГОР: ДЕТЕКЦІЯ, СЕГМЕНТАЦІЯ, РОЗУМІННЯ СЦЕН

Львів -- 2026

Лекція 8 (ч. 2): Комп'ютерний зір у розробці ігор — детекція, сегментація, розуміння сцен

Коли гра починає «бачити»: навіщо комп'ютерний зір у геймдеві

На попередніх заняттях ми навчили CNN розпізнавати зображення і навіть реконструювати 3D-сцени з фотографій. Але є фундаментальне питання, яке ми ще не розглядали з точки зору ігор: **а що, якщо сама гра — або її персонажі — повинні розуміти, що відбувається на екрані?**

Уявіть собі NPC-снайпера у тактичному шутері. Він стоїть на даху і дивиться на вулицю. Що він «бачить»? Для класичного ігрового AI — нічого візуального. Класичний бот працює з прямим доступом до ігрових даних: координати гравця, raycast-промені для перевірки видимості, заздалегідь розмічені навігаційні меші. Він «читерить» — отримує інформацію, яку людський гравець ніколи б не мав.

А тепер уявіть іншого бота. Він отримує лише те, що бачить гравець — **зображення з екрану**. І на основі цього зображення він повинен знайти ворога, оцінити відстань, прийняти рішення. Це і є задача комп'ютерного зору в контексті ігор: перетворити пікселі на розуміння.

Але комп'ютерний зір в іграх — це набагато більше, ніж зір NPC. Сьогодні CV використовується в геймдеві в трьох великих напрямках:

- **Всередині гри** — NPC, які «бачать» як гравець; системи, що аналізують ігрові кадри в реальному часі
- **У розробці** — автоматичне тестування, пошук багів на скріншотах, класифікація ігрового контенту
- **Навколо гри** — аналіз стрімів, античіт-системи, розпізнавання жестів гравця

На цій лекції ми пройдемо повний шлях: від базових задач детекції об'єктів до сучасних мультимодальних моделей, і на кожному кроці побачимо, як це працює в реальних іграх.

Детекція об'єктів: знайти і локалізувати

Що таке детекція

Класифікація відповідає на питання «що на зображенні?». Детекція об'єктів відповідає на більш конкретне: **«що і де?»** — для кожного знайденого об'єкта алгоритм малює обмежувальну рамку (bounding box) і присвоює клас із рівнем впевненості.

Формально, для кожного об'єкта на зображенні модель передбачає:

- координати рамки (x, y, w, h)
- клас $c \in 1, \ldots, C$
- впевненість $p(c) \in [0, 1]$

Це здається простою задачею, але складність у тому, що на одному зображенні може бути **десятки або сотні об'єктів різного розміру**, частково перекритих, на складному фоні. І все це потрібно знайти за один прохід мережі.

Еволюція підходів

Історія детекції об'єктів — це історія боротьби між точністю та швидкістю.

Двоетапні детектори (Two-stage). R-CNN (2014) першим показав, що CNN може детектувати об'єкти значно краще за класичні методи. Ідея: спочатку алгоритм Selective Search пропонує ~2000 «регіонів інтересу» (region proposals), потім CNN класифікує кожен. Проблема — катастрофічна повільність: секунди на одне зображення. Fast R-CNN (2015) прискорив класифікацію, обчислюючи CNN один раз для всього зображення. Faster R-CNN (2016) замінив Selective Search на **Region Proposal Network (RPN)** — маленьку мережу, що сама пропонує регіони. Faster R-CNN досяг ~5 FPS і став стандартом на кілька років.

Одноетапні детектори (One-stage). YOLO — You Only Look Once — радикально змінив підхід у 2016 році. Замість двох етапів, YOLO ділить зображення на сітку $S \times S$ і одночасно передбачає рамки та класи для кожної комірки. Один прохід мережі — і всі об'єкти знайдені.

YOLO: від v1 до v11

YOLO став, мабуть, найуспішнішою серією моделей у комп'ютерному зорі за останнє десятиліття. Кожна версія покращувала точність або швидкість:

Версія	Рік	Ключове нововведення
YOLOv1	2016	Одноетапна детекція на сітці
YOLOv2	2017	Anchor boxes, batch normalization
YOLOv3	2018	Multi-scale detection (FPN), ResNet-подібна архітектура
YOLOv4	2020	Bag of freebies (аугментації), CSPNet backbone
YOLOv5	2020	PyTorch-native, зручний API (Ultralytics)
YOLOv8	2023	Anchor-free, покращена архітектура голови
YOLOv11	2024	C3k2 блоки, ефективніший neck, підтримка edge

Сучасний YOLOv8/v11 досягає **mAP 53–55%** на COCO (80 класів) при швидкості **400+ FPS** на сучасному GPU. Для ігрових задач, де класів значно менше і зображення більш контрольовані, точність може бути значно вищою.

Метрика: mAP

Як виміряти якість детекції? Стандартна метрика — **mean Average Precision (mAP)**. Для кожного класу будується крива precision-recall: для різних порогів впевненості обчислюється, скільки знайдених об'єктів справді є об'єктами (precision) і скільки реальних об'єктів знайдено (recall). Площа під цією кривою — це Average Precision (AP) для одного класу. mAP — середнє AP по всіх класах.

Для визначення, чи знайдений об'єкт «правильний», використовується **IoU (Intersection over Union)** — відношення площі перетину передбаченої та реальної рамок до площі їх об'єднання:

$$\mathrm{IoU} = \frac{|B_{\mathrm{pred}} \cap B_{\mathrm{gt}}|}{|B_{\mathrm{pred}} \cup B_{\mathrm{gt}}|}$$

Зазвичай об'єкт вважається знайденим, якщо $\mathrm{IoU} \geq 0.5$ (mAP@50) або усереднюється по діапазону порогів від 0.5 до 0.95 (mAP@50:95).

Детекція об'єктів в іграх: реальні застосування

NPC з «справжнім» зором

Традиційний підхід до зору NPC в іграх — raycast: промінь від «очей» NPC до потенційних цілей, перевірка на перешкоди. Це дешево обчислювально, але створює неприродну поведінку: NPC або бачить ціль ідеально, або не бачить зовсім. Немає проміжних станів, немає помилок, немає поступового розпізнавання.

Дослідники з OpenAI, DeepMind та ігрових студій експериментують з іншим підходом: NPC отримує **рендер своєї точки зору** (від першої особи) і аналізує його нейромережею. Це дозволяє моделювати реалістичну поведінку:

- **Поступове розпізнавання** — NPC спочатку помічає рух, потім ідентифікує ціль. Як людина.
- **Обмежене поле зору** — NPC не «знає» про об'єкти поза кадром. Знову ж, як людина.
- **Помилки ідентифікації** — в тумані або темряві NPC може сплутати союзника з ворогом.
- **Контекстна обізнаність** — NPC може помітити не лише ворога, а й підозрілий предмет на столі, відкриті двері, розбите вікно.

Проєкт **MineRL** та середовище **MineClip** показали, що RL-агенти, натреновані на основі візуального входу в Minecraft, розвивають стратегії, подібні до людських — вони дивляться на горизонт, шукають ресурси поглядом, уникають монстрів, яких бачать.

Автоматичне тестування ігор (Visual QA)

Ігрові студії AAA-класу випускають ігри з тисячами локацій, персонажів, предметів. Ручне тестування кожної комбінації — непідйомне завдання. Комп'ютерний зір автоматизує цей процес:

- **Виявлення графічних артефактів** — детектор навчений знаходити текстурні глітчі, z-fighting (мерехтіння перекритих поверхонь), «провали» крізь геометрію
- **Перевірка UI** — автоматична верифікація, що елементи інтерфейсу не перекривають ігрову інформацію, текст читабельний, іконки на місцях
- **Порівняння версій** — автоматичне порівняння скріншотів між білдами для виявлення регресій

Electronic Arts використовує систему **AutoPlay**, де бот грає тисячі годин і детектор аналізує кожен кадр на наявність визначених категорій проблем. **Ubisoft** розробив систему **Ghostrunner QA**, де агенти проходять рівні і класифікують візуальні аномалії.

Античіт системи

Це, можливо, найбільш масштабне комерційне застосування CV в іграх. Античіт-системи аналізують поведінку гравців, але сучасні чіти стали настільки витонченими, що поведінковий аналіз недостатній. CV додає новий рівень:

- **Детекція підозрілих оверлеїв** — розпізнавання сторонніх елементів на екрані (прицільні сітки, ESP-рамки)
- **Аналіз руху прицілу** — чи рухається прицільна точка неприродно різко до цілей (aimbot)
- **Відеоаналіз стрімів** — системи автоматично аналізують стріми для виявлення читерства

Riot Games (Valorant) та Easy Anti-Cheat використовують ML-моделі, натреновані на мільйонах кліпів ігрового процесу, для виявлення аномальних патернів.

Аналіз ігрового процесу та кіберспорт

CV перетворює ігрові відеозаписи на структуровані дані:

- **Автоматична розмітка подій** — YOLO-подібна модель знаходить на кадрі позиції гравців, об'єктів, зброї. Це дозволяє автоматично генерувати статистику матчів без доступу до ігрового API.
- **Мінімапа → стратегічна аналітика** — детекція позицій гравців на мінімапі для побудови теплових карт руху, аналізу ротацій
- **Highlight detection** — автоматичне виділення ключових моментів (вбивства, клатчі, ейси) на основі візуального аналізу кадрів

Компанія **GRID** (раніше Bayes Esports) використовує CV для парсингу відеотрансляцій турнірів Counter-Strike та Dota 2, перетворюючи відео на структуровані дані для аналітики та ставок.

Сегментація: попіксельне розуміння

Три рівні сегментації

Якщо детекція малює рамки навколо об'єктів, сегментація йде далі — присвоює клас **кожному пікселю** зображення. Це більш точне і більш корисне розуміння сцени, але й обчислювально дорожче.

Існують три рівні сегментації:

Семантична сегментація (Semantic Segmentation) — кожен піксель отримує мітку класу, але окремі екземпляри одного класу не розрізняються. Якщо на зображенні три людини, всі їхні пікселі будуть позначені як «людина», без розрізнення хто є хто.

Інстансна сегментація (Instance Segmentation) — кожен окремий об'єкт отримує унікальну маску. Три людини — три окремі маски. Це комбінація детекції (знайти об'єкт) і сегментації (точна маска).

Паноптична сегментація (Panoptic Segmentation) — поєднує обидва підходи. Об'єкти, що піддаються підрахунку (люди, машини, тварини), отримують індивідуальні маски. Фонові класи (небо, дорога, трава) — лише семантичну розмітку.

Ключові архітектури

U-Net (2015) — архітектура, що стала золотим стандартом для семантичної сегментації. Ідея елегантна: **encoder** поступово зменшує просторове розрішення, витягуючи абстрактні ознаки. **Decoder** поступово відновлює просторову роздільність. Критичний елемент — **skip connections** між відповідними рівнями encoder та decoder, що дозволяють decoder використовувати деталі з ранніх шарів для точної локалізації меж.

DeepLab v3+ (2018) — використовує **atrous (dilated) convolutions** — згортки з «дірками», що збільшують рецептивне поле без збільшення кількості параметрів. Модуль **ASPP (Atrous Spatial Pyramid Pooling)** застосовує згортки з різними dilation rates паралельно, захоплюючи контекст на різних масштабах.

Mask R-CNN (2017) — розширення Faster R-CNN для інстансної сегментації. До кожної запропонованої рамки додається маленька FCN (Fully Convolutional Network), що передбачує бінарну маску об'єкта. Просте, але ефективне рішення.

SAM (Segment Anything Model) (2023, Meta) — революційна модель, натренована на 1.1 мільярді масок. SAM може сегментувати **будь-який об'єкт** на будь-якому зображенні без додаткового навчання, приймаючи як підказку точку, рамку або текстовий опис. Це змінило парадигму: замість тренування моделі для конкретних класів, SAM є «фундаментальною моделлю» для сегментації.

Сегментація в іграх: від пікселів до рішень

Процедурна генерація із реального світу

Один з найцікавіших сценаріїв — використання сегментації для створення ігрового контенту з фотографій реального світу:

1. Супутникове фото міста → семантична сегментація (будівлі, дороги, парки, вода)
2. Кожен клас → ігровий тип тайлу або біом
3. Результат — карта для стратегії або RPG, заснована на реальній географії

Гра **Cities: Skylines** використовує реальні дані висот для генерації карт. Додавання CV-сегментації дозволяє автоматично розмічувати зони (житлова, промислова, зелена) за супутниковим фото.

Цей самий підхід працює для ландшафтів: фотографія гірського масиву → сегментація (скелі, сніг, ліс, луки, вода) → автоматичне текстурування терену в Unreal Engine на основі масок.

Маттінг та ефекти в реальному часі

Ігри з камерою (Just Dance, Xbox Kinect ігри, AR-ігри на мобільних) потребують **відділення гравця від фону** в реальному часі. Це задача сегментації:

- **MediaPipe** (Google) — реалтайм-сегментація людини на мобільних пристроях, 30+ FPS на телефоні
- **Background matting** — використовується в AR-іграх (Pokemon GO, Minecraft Earth) для реалістичного розміщення віртуальних об'єктів

Навігація та прохідність

Для RL-агентів і ботів семантична сегментація — це спосіб зрозуміти структуру середовища безпосередньо з пікселів:

- Сегментація кадру від першої особи → маски «підлога», «стіна», «перешкода», «двері», «сходи»
- Агент приймає рішення на основі цих масок, а не на основі привілейованого доступу до геометрії рівня
- Проєкт **ViZDoom** (DOOM для навчання RL-агентів) демонструє агентів, що використовують семантичні маски для навігації рівнями

Аналіз ігрових сцен для дизайнерів

Сегментація допомагає аналізувати візуальну композицію ігрових кадрів:

- Яку частку екрану займає небо? Персонажі? UI? — для балансу візуальної композиції
- Чи достатньо контрастний головний персонаж на тлі середовища?
- Де гравець дивиться? (eye tracking + сегментація = розуміння, які елементи привертають увагу)

Vision Transformers: трансформери бачать

Від NLP до зору

У Лекції 7 ми бачили, що CNN — основний інструмент комп'ютерного зору. Але у 2020 році дослідники з Google показали, що **Transformer** — архітектура, створена для обробки тексту — може працювати з зображеннями не гірше, а іноді й краще за CNN.

Vision Transformer (ViT) розрізає зображення на патчі (наприклад, 16×16 пікселів), перетворює кожен патч на вектор (embedding), додає позиційне кодування і пропускає через стандартний Transformer encoder. Ніяких згорток — лише self-attention.

Це працює тому, що self-attention дозволяє кожному патчу «спілкуватися» з усіма іншими патчами на зображенні одночасно. CNN бачить лише локальне оточення (рецептивне поле), а ViT має **глобальний контекст** з першого шару.

Swin Transformer

Проблема ViT: self-attention має квадратичну складність $O(n^2)$ відносно кількості патчів. Для зображень високої роздільності це занадто дорого. **Swin Transformer** (2021, Microsoft) розв'язує це через **вікна** (windows): self-attention обчислюється лише всередині локальних вікон, а між вікнами — через зсунуті вікна (shifted windows). Це дає лінійну складність і ієрархічне представлення, подібне до CNN.

Swin Transformer став новим backbone для детекції та сегментації, перевершивши CNN на більшості бенчмарків.

DETR: End-to-End детекція

DETR (Detection Transformer, Facebook, 2020) — революційний підхід до детекції. Замість anchor boxes, NMS (Non-Maximum Suppression) і інших рукотворних компонентів, DETR формулює детекцію як **задачу множин**: Transformer decoder отримує фіксовану кількість «запитів» (object queries) і для кожного передбачує або об'єкт, або «порожній клас». Угорський алгоритм (Hungarian matching) забезпечує унікальне зіставлення передбачень з реальними об'єктами.

Елегантність DETR — у простоті: жодних anchor boxes, жодного NMS, жодних рукотворних порогів. Архітектура повністю диференційовна, end-to-end.

Чому це важливо для ігор

Vision Transformers особливо цікаві для ігрових застосувань через кілька властивостей:

- **Глобальний контекст** — для розуміння ігрової сцени важливо бачити загальну картину (хто де стоїть, який етап гри), а не лише локальні деталі
- **Гнучкість вхідних даних** — ViT може приймати патчі різного розміру, що дозволяє працювати з різними роздільностями
- **Мультимодальність** — трансформерна архітектура природно поєднується з текстовими моделями (CLIP, GPT-4V), що відкриває шлях до NPC, які розуміють і бачать, і говорять

CLIP: коли зір зустрічає мову

Ідея: спільний простір для зображень і тексту

CLIP (Contrastive Language–Image Pre-training, OpenAI, 2021) — це модель, що навчилася зв'язувати зображення та текст. Тренована на 400 мільйонах пар «зображення + підпис» з інтернету, CLIP створює **спільний embedding-простір**, де семантично близькі зображення та тексти мають близькі вектори.

Як це працює:

1. **Image encoder** (ViT або ResNet) перетворює зображення у вектор
2. **Text encoder** (Transformer) перетворює текст у вектор
3. Під час тренування CLIP максимізує схожість правильних пар «зображення-текст» і мінімізує схожість неправильних

Революційна властивість CLIP — **zero-shot** класифікація. Модель може класифікувати зображення за категоріями, яких ніколи не бачила під час тренування. Достатньо сформулювати категорії як текстові описи: «a photo of a sword», «a photo of a shield», «a photo of a potion» — і CLIP визначить, що зображено.

CLIP в ігровій розробці

Пошук і каталогізація ігрових ассетів. Ігрова студія з бібліотекою на 50 000 3D-моделей і текстур може використовувати CLIP для семантичного пошуку: художник набирає «mossy stone wall in forest» і отримує релевантні ассети. Це вже працює в **Shutterstock** та **Adobe Stock** для фотографій, і активно впроваджується для 3D-контенту.

Автоматична класифікація ігрового контенту. CLIP може визначати: жанр скріншоту (шутер, RPG, гонки), настрій сцени (темний, яскравий, тривожний), наявність конкретних елементів (зброя, кров, текст) — все це без спеціального тренування під конкретну гру.

Генерація з текстового опису. CLIP став ключовим компонентом систем генерації зображень (DALL-E, Stable Diffusion). Для ігор це означає: дизайнер описує текстом «abandoned space station corridor with flickering lights» → генеративна модель створює концепт-арт або текстуру.

Валідація стилістичної консистентності. CLIP може оцінити, наскільки новий ассет відповідає встановленому арт-стилю гри, порівнюючи embeddings нового контенту з еталонними.

SAM та інтерактивна сегментація: новий інструмент для художників

Segment Anything Model

SAM (Meta, 2023) заслуговує окремої уваги, тому що він змінив робочий процес для ігрових художників і технічних дизайнерів. SAM — це «foundation model» для сегментації, натренований на **SA-1B** — датасеті з 11 мільйонів зображень та 1.1 мільярда масок.

SAM працює як інтерактивний інструмент: ви даєте підказку (клік, рамка, текст), а модель сегментує відповідний об'єкт. Ключові властивості:

- **Нульове навчання** — працює на будь-яких зображеннях, включаючи ігрові скріншоти, без fine-tuning
- **Інтерактивність** — один клік на персонажа → маска персонажа; рамка навколо групи → маски всіх об'єктів у рамці
- **Ієрархічність** — SAM може виділити і цілу будівлю, і окреме вікно, і фрагмент рами вікна — залежно від рівня деталізації

Застосування в пайплайні розробки

Швидке створення масок для навчальних даних. Раніше для тренування моделі під конкретну гру потрібно було вручну розмічати тисячі зображень. SAM скорочує цей процес на порядок: один клік на об'єкт → готова маска.

Вирізання об'єктів для маркетингу. Промо-арт вимагає ізоляції персонажів, зброї, середовищ зі скріншотів. SAM автоматизує те, що раніше робилося годинами в Photoshop.

Автоматична сегментація для LOD. При створенні LOD (Level of Detail) потрібно визначити, які частини моделі візуально важливі. SAM на рендері моделі може виділити ключові візуальні елементи.

SAM 2: відео

SAM 2 (Meta, 2024) розширив можливості на відео: виділяєте об'єкт на одному кадрі — модель відстежує його на всіх наступних кадрах. Це відкриває можливості для:

- Автоматичного трекінгу ігрових об'єктів у відео для аналітики
- Створення масок для VFX у кінематичних роликах
- Відстеження руху гравців для аналізу кіберспортивних матчів

Оцінка глибини: 3D з одного фото

Monocular Depth Estimation

Окремий, але надзвичайно важливий напрямок CV — **оцінка глибини з одного зображення**. На відміну від SfM, який потребує набору фотографій з різних ракурсів, monocular depth estimation передбачає карту глибини з **одного знімку**.

Як це можливо? Мережа навчилася на мільйонах зображень з відомою глибиною (від LiDAR, стереокамер) розпізнавати візуальні підказки: перспективу, розмір відомих об'єктів, текстурні градієнти, атмосферну перспективу (далекі об'єкти більш розмиті та блакитні).

Depth Anything (2024)

Depth Anything (TikTok/ByteDance, 2024) — state-of-the-art модель для моноколярної оцінки глибини. Натренована на 62 мільйонах зображень, вона видає якісну відносну карту глибини для будь-якого зображення за кілька мілісекунд. **Depth Anything v2** покращила точність і додала метричну глибину (абсолютні відстані в метрах).

Застосування в іграх

Створення нормал-мепів і карт висот. 2D-художник малює текстуру → Depth Anything генерує карту глибини → автоматично створюється normal map для імітації 3D-рельєфу. Це замінює годинну ручну роботу в Substance Designer.

2.5D ефекти з 2D-арту. Фотографія або малюнок → карта глибини → паралакс-ефект, який створює ілюзію глибини. Використовується в мобільних іграх та UI для створення «живих» фонів.

Автоматичний Depth of Field. Замість симуляції DoF через рендеринг (дорого), карта глибини від нейромережі дозволяє застосувати розмиття в пост-обробці на основі «реальної» глибини.

Автоматична окклюзія в AR. В AR-іграх (Pokemon GO) віртуальні об'єкти повинні правильно перекриватися реальними об'єктами. Карта глибини від камери телефону дозволяє Pokemon «ховатися» за деревами і будівлями.

Комп'ютерний зір в реальних іграх: кейс-стаді

Кейс 1: NPC-зір в MineRL та MineDojo

MineRL — середовище для тренування RL-агентів у Minecraft, де агент отримує лише пікселі (кадр 64×64 від першої особи) і команди клавіатури/миші. Жодного привілейованого доступу до ігрових даних.

MineDojo (2022, NVIDIA) розширив цей підхід: 730 000 відео з YouTube + 7 000 wiki-сторінок Minecraft використані для тренування мультимодальної моделі **MineCLIP**, яка зв'язує відео геймплею з текстовими описами. Агент може виконувати текстові інструкції: «build a house near water» — і виконує їх, розуміючи середовище через зір.

Це прототип майбутнього NPC: він не має привілейованої інформації, він «бачить» світ як гравець і діє на основі розуміння зображення.

Кейс 2: Visual QA в Assassin's Creed

Ubisoft La Forge розробила систему автоматичного тестування для серії Assassin's Creed. Бот-тестувальник:

1. Проходить ігрові рівні за допомогою навігаційних агентів
2. На кожному кроці робить скріншот і аналізує його:
    - Детекція: чи є на кадрі NPC, що «застрягли» в геометрії?
    - Сегментація: чи правильно рендериться вода, небо, рослинність?
    - Класифікація: який тип локації (місто, ліс, печера)?
3. Аномалії автоматично логуються як баг-репорти з координатами та скріншотом

Система знайшла тисячі багів, які були б пропущені при ручному тестуванні.

Кейс 3: Розпізнавання жестів для ігрового вводу

Ігри з камерою (Just Dance, Beat Saber з hand tracking, Nintendo Ring Fit) використовують CV для розпізнавання рухів тіла як ігрового вводу:

- **MediaPipe Pose** (Google) — реалтайм-детекція 33 ключових точок тіла
- **MediaPipe Hands** — 21 точка на кожній руці для жестового управління
- **Apple Vision Pro** — трекінг рук і погляду як основний ввід для ігор у mixed reality

Це приклад, де CV працює не всередині гри, а **на межі між гравцем і грою**, перетворюючи фізичні рухи на ігрові команди.

Кейс 4: Стилізація та генерація контенту

Neural Style Transfer та його нащадки використовуються для автоматичної стилізації ігрових ассетів:

- Фотографія → стилізація під конкретний арт-стиль гри (cel-shading, пікселарт, аквакерль)
- **GauGAN / SPADE** (NVIDIA) — малюєте грубий скетч з кольоровими зонами → модель генерує фотореалістичний ландшафт
- **ControlNet + Stable Diffusion** — ігровий скріншот → перегенерація в іншому стилі зі збереженням композиції

Практичні аспекти: швидкість і обмеження

Час інференсу: що реально працює в реальному часі?

Для ігор час — критичний ресурс. Бюджет одного кадру при 60 FPS — 16.7 мс, з яких на CV можна відвести лише частину. Ось реалістичні показники:

Модель	Задача	Час (GPU)	Час (мобільний)	Реалтайм у грі?
YOLOv8-nano	Детекція	1.2 мс	8 мс	Так
YOLOv8-large	Детекція	5.8 мс	45 мс	Умовно
MobileNet-v3 + DeepLab	Сегментація	4 мс	25 мс	Так (мобільні)
SAM (ViT-H)	Інтерактивна сегм.	50 мс	—	Ні (інструмент)
Depth Anything v2 (small)	Глибина	3 мс	15 мс	Так
CLIP (ViT-B/32)	Embedding	4 мс	30 мс	Оффлайн

Висновок: легкі моделі (YOLO-nano, MobileNet, Depth Anything-small) можуть працювати в реальному часі навіть на мобільних пристроях. Важкі моделі (SAM, великі ViT) підходять для інструментів розробки та офлайн-аналітики.

Оптимізація для ігрових рушіїв

- **ONNX Runtime** — експорт моделі в ONNX-формат для ефективного інференсу на різних платформах
- **TensorRT** (NVIDIA) — оптимізація моделі під конкретний GPU: квантизація (FP16, INT8), злиття операцій, оптимізація графа обчислень
- **Core ML** (Apple) — для iOS/macOS ігор, з апаратним прискоренням на Neural Engine
- **Barracuda** (Unity) — бібліотека для інференсу нейромереж всередині Unity, підтримує ONNX

Типовий пайплайн інтеграції

```
Тренування (офлайн)              Інференс (у грі)
──────────────────               ────────────────
PyTorch модель                   ONNX / TensorRT модель
   ↓                               ↓
torch.export → ONNX              Ігровий рушій (C++)
   ↓                               ↓
TensorRT optimize                GPU інференс (< 5 мс)
   ↓                               ↓
Деплой у білд                    Результат → ігрова логіка
```

Повний пайплайн розуміння сцени для гри

Окремі задачі CV — детекція, сегментація, оцінка глибини — потужні самі по собі. Але справжня сила з'являється, коли вони працюють **разом**. Ось як виглядає повний пайплайн «розуміння сцени» для ігрового AI або аналітичної системи:

```
Ігровий кадр (RGB)
       ↓
┌──────────────────────────────────────────┐
│  Паралельна обробка:                     │
│  ┌─────────┐ ┌──────────┐ ┌──────────┐  │
│  │ YOLO    │ │ DeepLab  │ │ Depth    │  │
│  │ Детекція│ │ Сегмент. │ │ Anything │  │
│  └─────────┘ └──────────┘ └──────────┘  │
│       │           │           │          │
│       ▼           ▼           ▼          │
│  Об'єкти +     Семантична    Карта       │
│  рамки + класи  карта сцени  глибини     │
└──────────────────────────────────────────┘
       │           │           │
       ▼           ▼           ▼

┌──────────────────────────────────────────┐
│           Fusion / Інтеграція            │
│  • 3D-позиції об'єктів (детекція+глибина)│
│  • Прохідна зона (сегментація+глибина)   │
│  • Контекст сцени (CLIP embedding)       │
└──────────────────────────────────────────┘
                   │
                   ▼
         Ігрова логіка / RL-агент
```

Цей пайплайн дозволяє RL-агенту або NPC мати повне «розуміння» ігрової сцени, побудоване виключно з пікселів — без привілейованого доступу до внутрішніх даних гри.

Підсумок

Ключові ідеї лекції

1. **Детекція (YOLO)** знаходить об'єкти і їх позиції — основа для NPC-зору, автоматичного QA, античіту, кіберспортивної аналітики.

2. **Сегментація (U-Net, DeepLab, SAM)** дає попіксельне розуміння сцени — основа для процедурної генерації, AR-ефектів, навігації агентів.

3. **Vision Transformers (ViT, Swin, DETR)** перенесли архітектуру трансформерів у зір, даючи глобальний контекст і уніфіковану архітектуру для всіх задач CV.

4. **CLIP** з'єднав зір і мову, відкривши zero-shot класифікацію, семантичний пошук ассетів і генерацію з текстових описів.

5. **Оцінка глибини (Depth Anything)** дає 3D-розуміння з одного фото — для AR, нормал-мепів, пост-обробки.

6. **У реальних іграх** CV працює на трьох рівнях: всередині гри (NPC-зір, RL-агенти), у розробці (тестування, інструменти), навколо гри (античіт, аналітика, ввід).

Зв'язок з іншими лекціями

- **Лекція 7 (CNN)** — архітектури, які є backbone для всіх моделей цієї лекції
- **Лекція 8 ч.1 (SfM)** — 3D-реконструкція, де CV-моделі (детекція ознак, matching) є першими кроками
- **Лекції 12–14 (RL)** — агенти, що використовують CV як «очі» для навчання в ігрових середовищах
- **Лекція 15 (генеративні моделі)** — CLIP і сегментація як компоненти генеративних пайплайнів

Що далі

На наступних лекціях ми перейдемо до **обробки послідовностей** — RNN, LSTM, механізм уваги та архітектура Transformer, що стала основою не лише Vision Transformers, але й великих мовних моделей.